Function in C

Function

- **Definition**: A function is a self-contained block of code that performs a specific task.

- **Why Use Functions?**
  - Modularize code for easier management
  - Reusable blocks to avoid code duplication
  - Simplifies debugging and testing

- **Example**: A function that calculates the sum of two numbers.

Basic Syntax:

```
return_type function_name(parameter_list)
 {
   // function body
   return value; // if return_type is not void
}
```

- **Return Type** : Type of data returned by the function (e.g., `int`, `float`, `void`)
- **Function Name** : Unique identifier for the function
- **Parameter List** : Variables to pass values to the function
- **Function Body** : Statements defining what the function does
- **Return Statement**: (if needed) specifies the output of the function

Function Declaration

- To declare a function, we should specify the function return type, function name and function parameters.

Syntax:

```
return_type function_name(parameters);
```

Return type

How to select the return type?

- When we don't want to return any value from the function, we should declare return type as `void`.

Example:

```
void function_name(parameters);
```

- When we want to return an **integer** value from a function, declare return type as `int`.
- Similarly, for `float`, `char` and `double`.

Example:

```
int    function_name(parameters);
char   function_name(parameters);
float  function_name(parameters);
double function_name(parameters);
```

Contd...

Function name

- We should give a meaningful function name. After function name we should append open and close bracket ( ).
- If we want to write a program to add two numbers, we can declare function name as add or sum like below.

```
Example:

return_type add( parameters );
return_type sum( parameters );
```

Parameters

- Function parameter refers the input which are passed while calling the function.
- we have to specify the parameters list inside the bracket ().

Contd...

Example

- Let's declare a function which will have two integer input parameters and returns the sum of two integers.
- sum will be a integer value, so declare return type as **int**.
- we are going to calculate the sum of two numbers, so we can name it **add or sum**.
- parameters will be two integers, so we can declare a function prototype like below,

```
int add(int num1, int num2);
```

- parameter name is optional one but datatype of the parameter is mandatory.
- So, we can also declare the same prototype like this,

```
int add(int, int);
```

- we can name the parameter anything while defining the function.
- Parameter field is an optional one.
- we can also declare a function without any parameter like this,

```
int fun();
```

Function Definition

- In the function definition, we will implement the actual task of a function using a group of statements.

Write a function to add two numbers and return their sum.

Example:

```
int sum(int a, int b)
{
    int ans;
    ans = a + b;
    return ans;
}
```

We can also write the above function without using temporary variable **ans** like below,

```
int sum(int a, int b)
{
    return a+b;
}
```

Difference between Function Declaration & Function Definition

Function Declaration

- In the function declaration, we are giving information about the function to the compiler.

```
Example:
    int sum(int,int);
```

- Function `sum` will return a integer value.
- And it will take two parameters both of them are type integer.

Function Definition

- In the function definition, we are actually implementing the function.
- In other words, we are defining the task using a group of statements.

```
Example:
    int sum(int a, int b)
    {
        int ans;
        ans = a + b;
        return ans;
    }
```

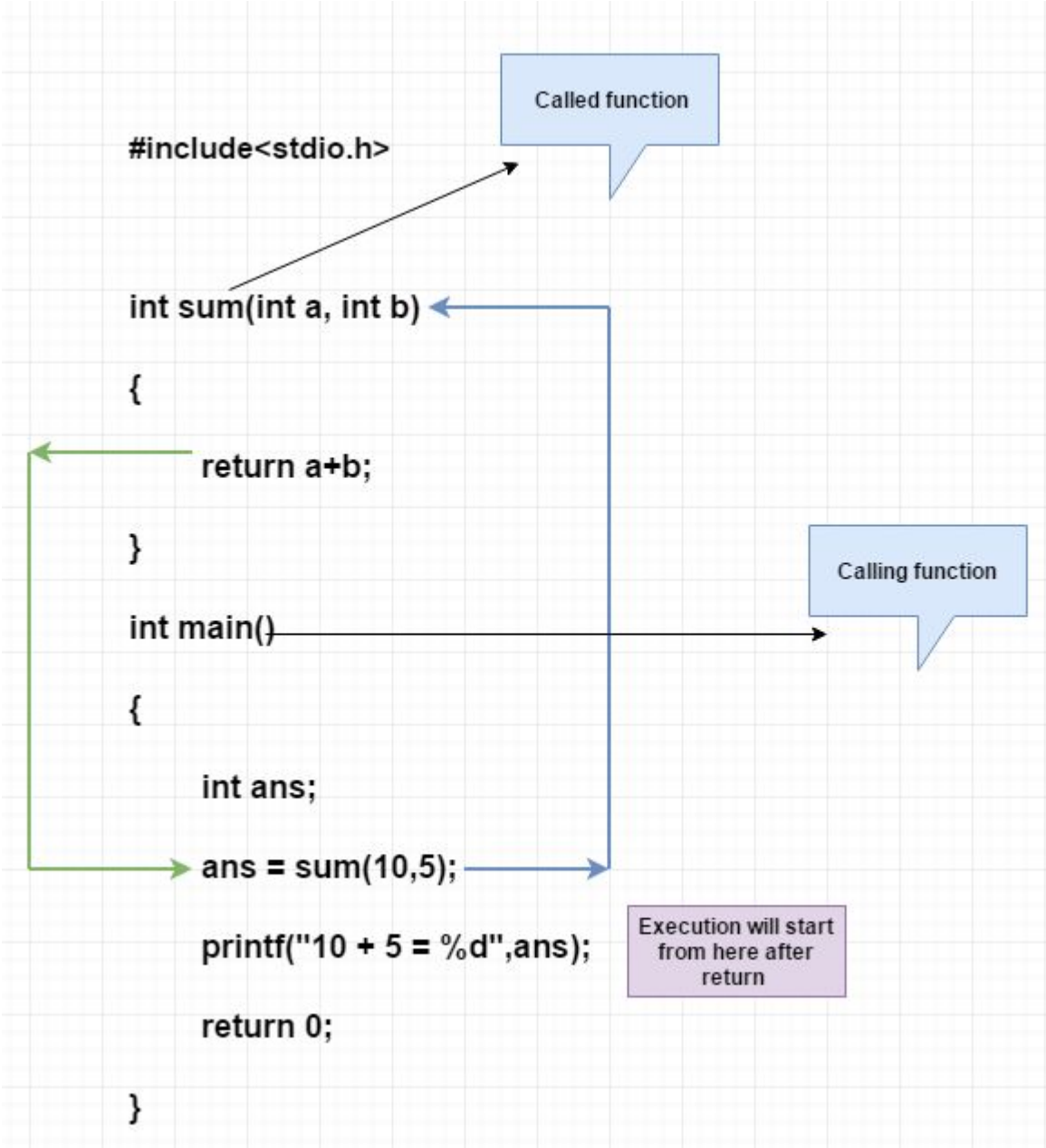
- Here, we are implementing addition task using a set of c statements.

return Statement

- When program execution reaches the **`return`** statement, it will leave the current function and the program execution will resume to the next statement, where it was actually called.

```
Example:

    return (value or variable);
    return 10;
    return a;
    return a+b;
```

- The value will be passed back to the function where it was called.
- We can receive the value and do the further programming from that point.



Where,

- **Calling function** - Main function (the point from where the function is called)
- **Called function** - the actual function which going to execute.
- **ans** will receive the sum result that is 15. in other words, ans will hold value 15 after the execution of return statement.
- And the program execution will resume from printf("10 + 5 = %d",ans); statement.

What will happen if we write some statements below to the return statement?

- If the program execution reaches return statement, the statements below to that will not be executed.
- Execution control will leave the current function immediately.

```c
#include<stdio.h>

int sum(int a, int b)
{
    return a+b;
    /*
     * printf will not be executed,
     * as it is below to the return statement
     */
    printf("Hello");
}
```

```c
int main()
{
    int ans;
    ans = sum(10,5);
    //ans will have the return value i.e a+b
    printf("10 + 5 = %d",ans);
    return 0;
}
```

- In the above program, `printf("Hello");` will not be executed as it's below to the return statement.

Types of Functions in C

Standard Library Functions: Built-in, such as `printf`, `scanf`, `sqrt`, etc.

User-Defined Functions: Functions that the programmer writes

Types by Return Value:

- Functions with return values
- Functions without return values (`void`)

Declaring and Defining Functions

Function Declaration (Prototype) :

A function declaration, also known as a ***function prototype***, is a statement that informs the compiler about the function's return type, name, and the types of its parameters. It doesn't include the actual code of the function.

```
int add(int, int);
```

Function Definition:

The function definition provides the full implementation of the function.

```
int add(int a, int b)
{
    return a + b;
}
```

Example

```c
#include <stdio.h>

// Function declaration
int add(int, int);

int main()
{
  int result;
  result = add(5, 3);  // Function call
  printf("Sum: %d\n", result);
  return 0;
}

// Function definition
int add(int a, int b)
{
  return a + b;
}
```

Example

```c
#include <stdio.h>

// Function definition
int add(int a, int b)
{
  return a + b;
}

int main()
{
  int result;
  result = add(5, 3);  // Function call
  printf("Sum: %d\n", result);
  return 0;
}
```

Function with Parameters

```
#include<stdio.h>
 void add ( int a , int b )
 {
    int sum = a + b ;
    printf ( " Sum : %d \n ",sum) ;
 }
 int main ()
 {
    add(5 , 3); // Output : Sum : 8
    return 0;
 }
```

Function with Return Value

```
#include<stdio.h>
int multiply ( int a , int b )
{
   return a * b ;
}
int main ()
{
 int result = multiply (4 , 5) ;
 printf ( " Product : %d \n " , result ) ; // Output : Product:20
 return 0;
}
```

Recursive Function

Recursive Function: A function that **calls itself** is called a recursive function.

```
#include<stdio.h>
 int factorial ( int n )
 {
    if ( n <= 1)
    {
        return 1;
    }
    return n * factorial ( n - 1) ;
 }

 int main ()
 {
    int num = 5;
    printf ( " Factorial of %d is %d \n " , num , factorial ( num ) ) ;
    return 0;
 }
```

Math function

```
#include<stdio.h>
#include<math.h>
int main()
{
double number = 16.0;
double result; // Square root function
result = sqrt(number);
printf("The square root of %f is %f\n", number, result);
return 0;
}
```

Math function

```
#include<stdio.h>
#include<math.h>
int main()
{
double base = 2.0;
double exponent = 3.0, result; // power function
result = pow(base, exponent);
printf("%.2f raised to the power of %.2f is %.2f\n", base, exponent, result);
return 0;
}
```

String in C

2 String

Why do we need Strings?

- Using **char** datatype, we can store a single character.

```
Example:

char c1 = 'a';   //characters are enclosed by single quotes
char c2 = '*';
```

How can we store a word or line?

- Like "india", "java programming" etc.
- We can store sequence of characters using **string**.

What is String?

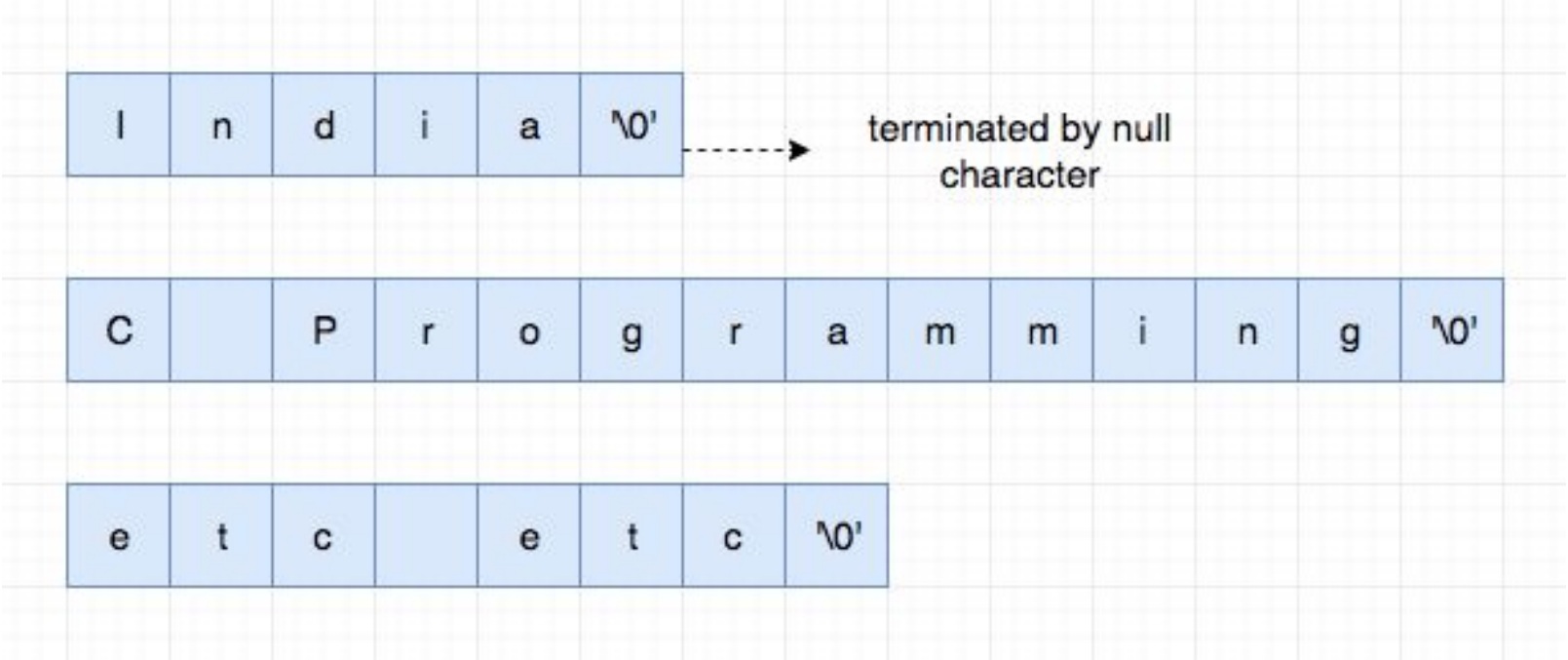
- String an array of characters terminated by null character ‘\0’.
- Strings are enclosed by **double quotes** whereas characters are enclosed by single quotes.

Example:

"India"

"C programming"

"etc etc"



- The string must be terminated by the null character '\0'.
- '\0' character denotes the end of a string which will be useful when we process the string.

Declaration of String

- To declare a string,

Syntax:

```
char variable_name[size];
```

Example:

```
char name[10];
char city[20];
```

Initialization of String

- There are multiple ways we can initialize a string.

Direct Initialization

- If we assign string directly with in double quotes, no need to bother about null character.
- Because compiler will automatically assign the null character at the end of string.

i) Assigning direct string with size

```
Example:

   char country[6] = "india";
```

ii) Assigning direct string without size

```
Example:

   char country[] = "india";
```

- while giving **initial size**, we should always give **extra one size** to store **null character**.
- To store "india", string **size 5** is enough but we should give extra one (+1) **size 6** to store null character.
- In general to store **N** character string, we should create **N+1** size char array.
- **In second case**, string size will be determined by compiler.

Contd...

Character by character Initialization

- we can also assign a string character by character.
- If we assign character by character , we must specify the null character '\0' at the end of the string.

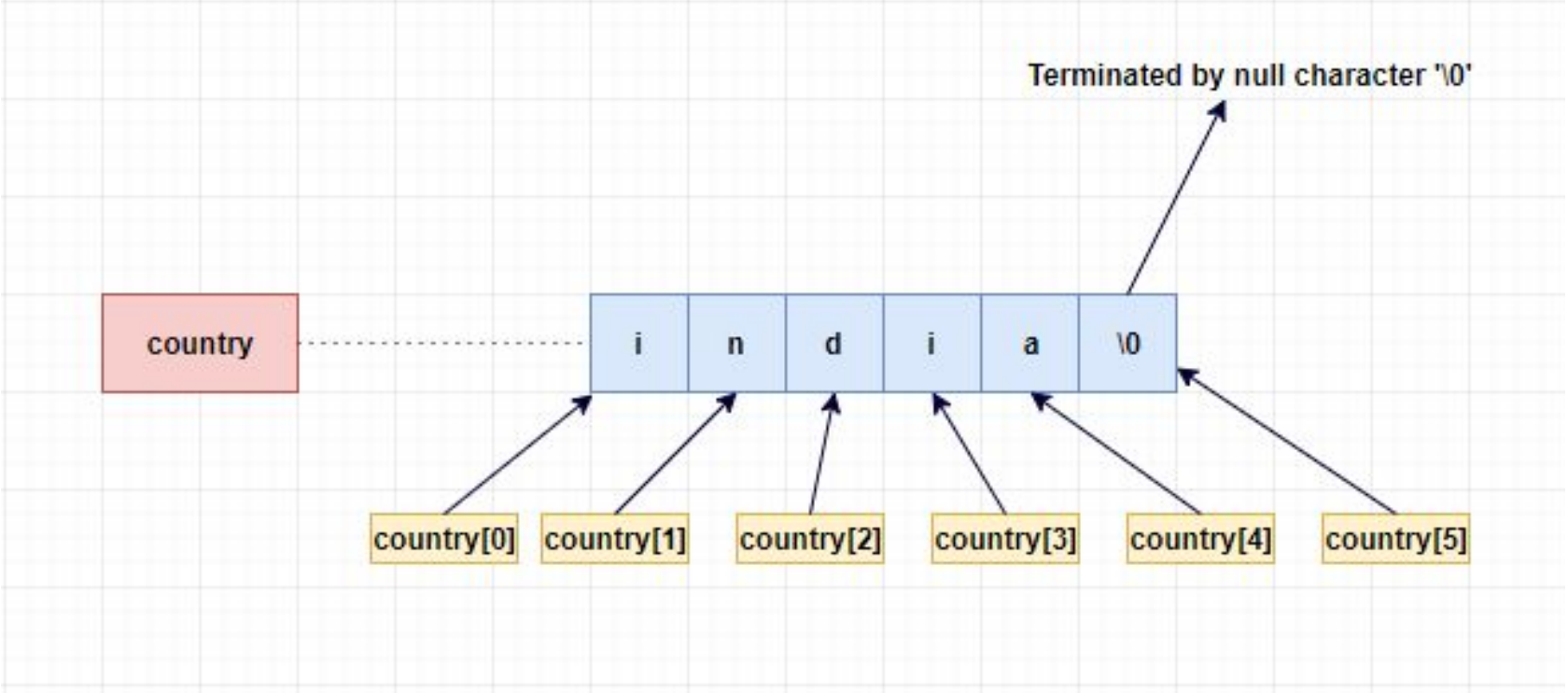
i) char by char with size

```
Example:

char country[6]={'i','n','d','i','a','\0'};
```

ii) char by char without size

```
Example:

char country[]={'i','n','d','i','a','\0'};
```

Contd…

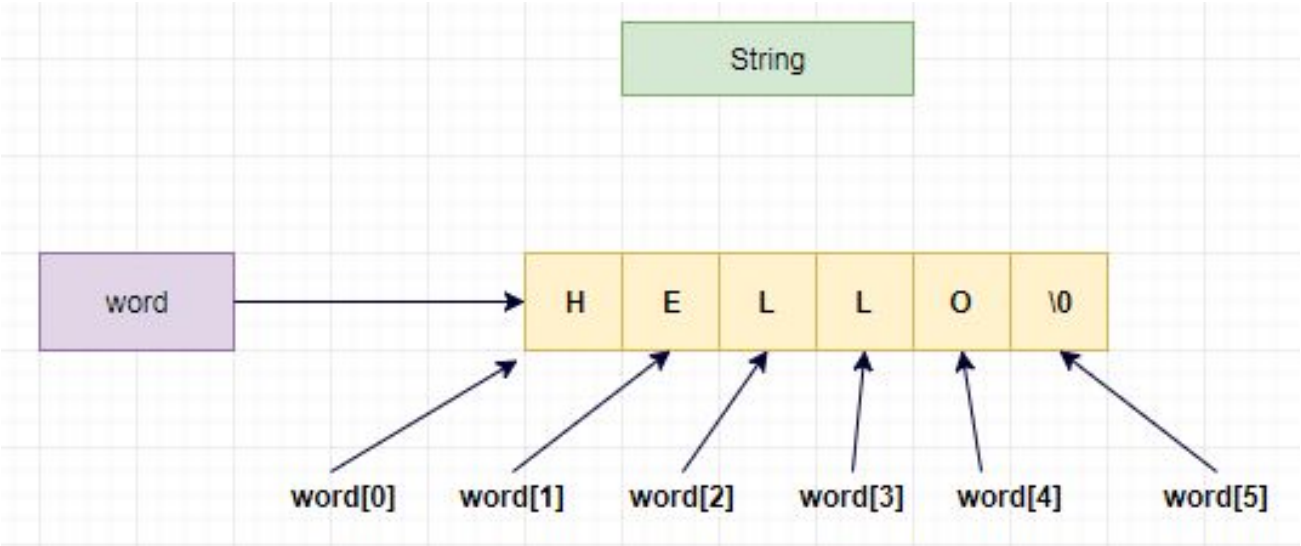


- All four methods are perfectly valid.

String Manipulation

- Let's take a sample string.

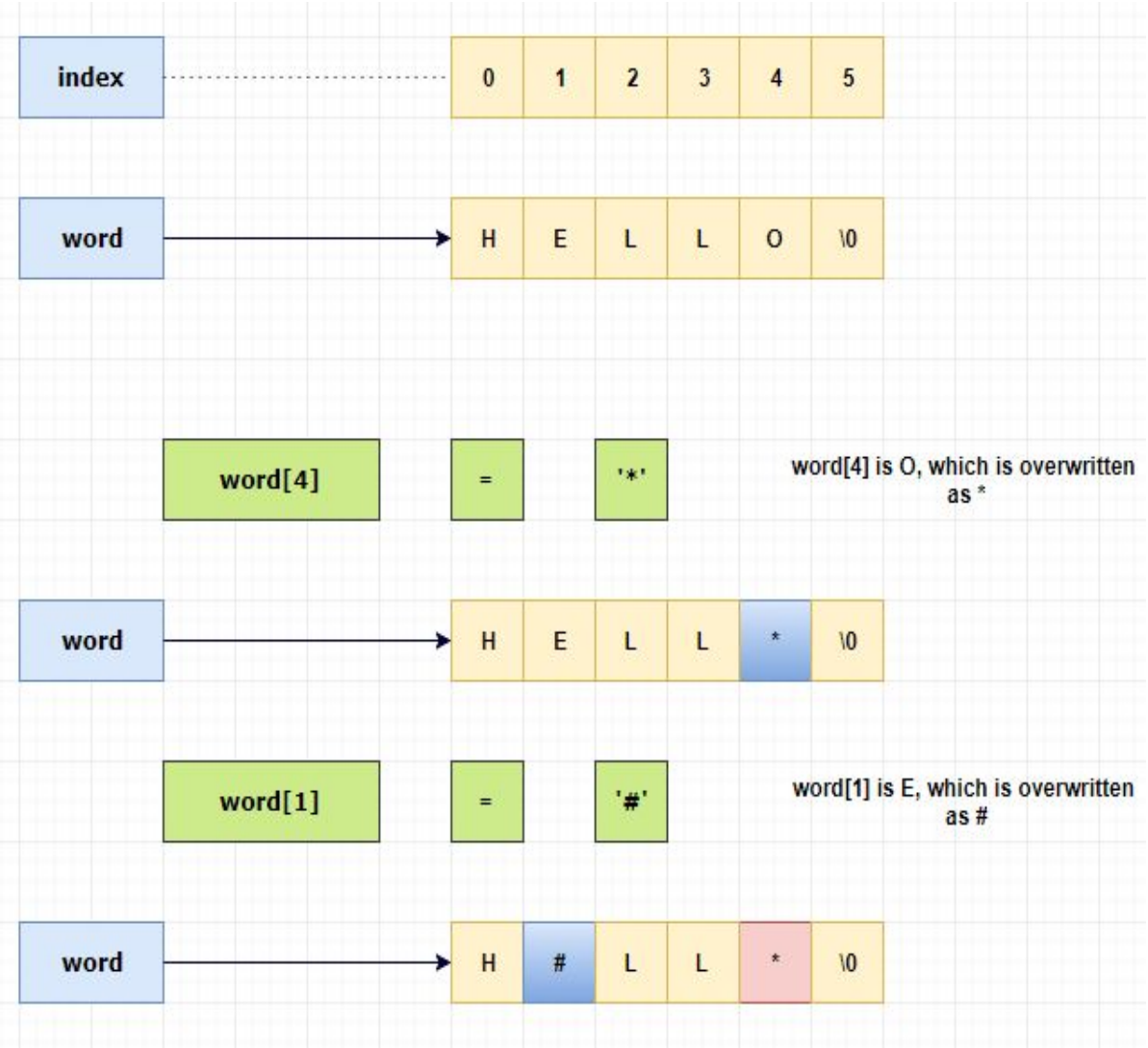
```
Example:

char word[6]="HELLO";
```



- We can access each character in the string using index value like array.
- String index also starts with 0.
- Nth character of the string can be accessed by using index N-1.
- Because string index starts with 0 Not 1.
- For Example, To access 1st character of word, we should use index 0.
- Similarly, To access 4th character of word, we should use index 3.

Program

```c
#include<stdio.h>
int main()
{
    char word[6]="HELLO";

    printf("%s\n",word);//printing unchanged string

    word[4] = '*';     //changing word index 4 as '*'.
    printf("%s\n",word);   //It will print "HELL*"

    word[1] = '#';     //changing word index 1 as '#'.
    printf("%s\n",word);   //It will print "H#LL*"

    return 0;
}
```



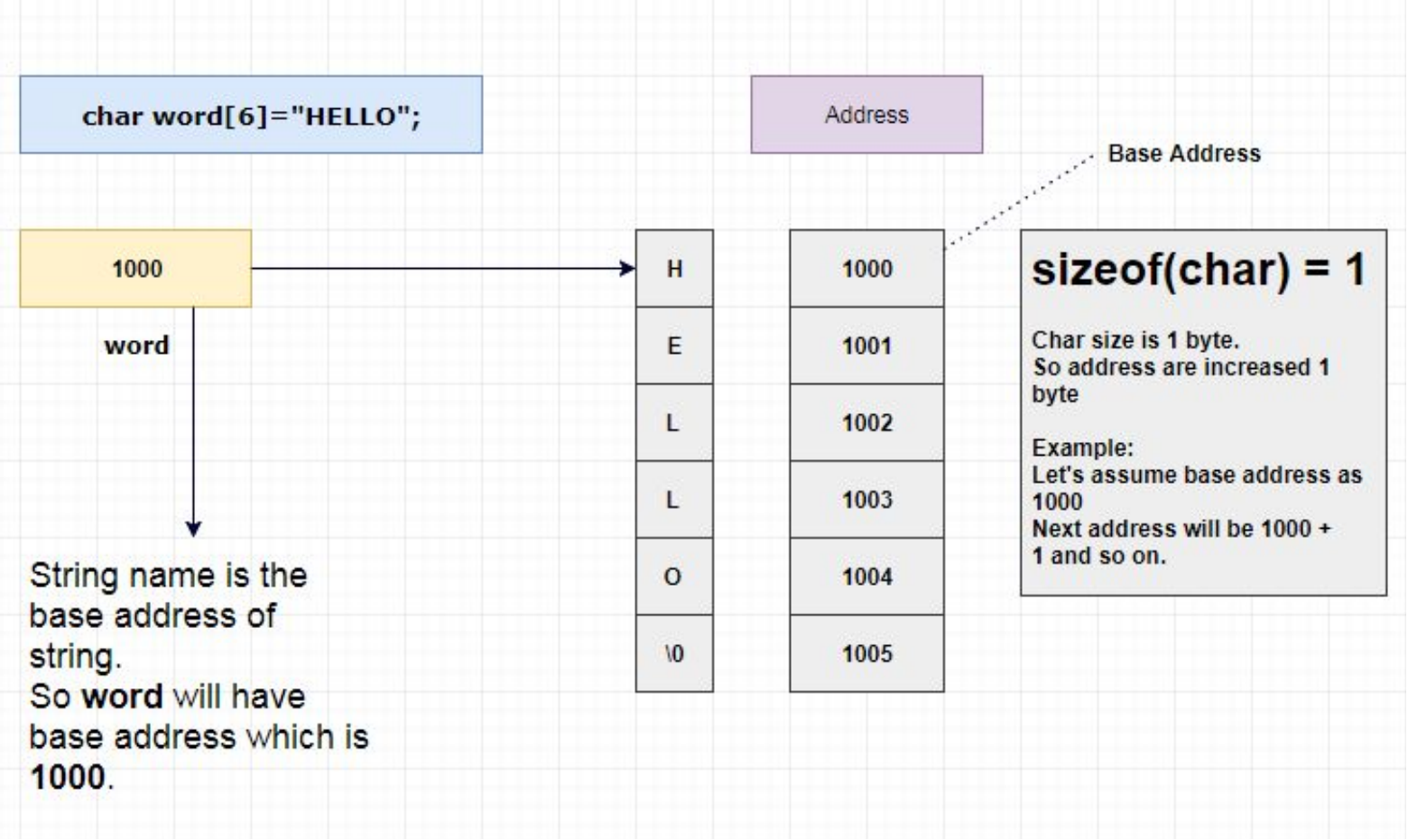
Getting String Input from user

- Using **%s** format specifier in **scanf**, we can get string input from the user.

Example:

```
char name[20];
scanf("%s",name);
```

- In scanf we didn't give **&name**. Why?
- Because string **name** itself base address of a string.
- So, no need to give **& operator** in scanf while getting string input.



Program using scanf()

```c
#include<stdio.h>

int main()
{
    char name[10];

    printf("Enter your name\n");

    scanf("%s",name);

    //print the name
    printf("Welcome %s\n",name);

    return 0;
}
```

%S in printf

- If we give **%s** to print a string,
- It will print every character of a string one by one until it encounters null character '**\0**'.
- If it hits the null character, it will stop printing characters.
- So, null character is very important in string. Otherwise, it will keep on print data which may show some sensitive data to attackers.

Program using fgets()

```
#include<stdio.h>

int main()
{
   char name[20];
   printf("Enter your name\n");

   fgets(name, sizeof(name), stdin);

   //print the name
   printf("Welcome %s\n",name);

   return 0;
}
```

//Program to find the length of an entered string

```c
#include<stdio.h>

int main()
{
   char name[20],i = 0, count = 0;
   printf("Enter your name\n");
   fgets(name, sizeof(name), stdin);
   while(name[i] != '\0' && name[i] != '\n')
   {
       count++;
       i++;
    }
   //print string length
   printf("The count is: %d\n",count);
  return 0;
}
```

When the user presses **Enter** after typing their input, **`fgets()`** includes the newline character (**`\n`**) in the array.